

**Московский государственный технический  
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»  
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»  
Отчет по ЛР №5-6. Python

Выполнила:  
Студент группы ИУ5-31Б  
Сербенюк Полина  
Проверил:  
Гапанюк Ю. Е.

2025 г.

Для лабораторной работы по шаблонам проектирования реализованы 3 шаблона из разных категорий на примере системы обработки заказов в интернет-магазине.

### **adapter.py**

```
class OldPaymentSystem:
```

```
    def make_payment(self, customer_code: str, amount_in_cents: int) -> dict:
        return {
            "status": "success",
            "transaction_id": f"OLD_{customer_code}_{amount_in_cents}",
            "amount": amount_in_cents / 100
        }
```

```
class PaymentAdapter:
```

```
    def __init__(self, old_system: OldPaymentSystem):
        self.old_system = old_system
```

```
    def pay(self, order_id: str, amount: float) -> dict:
```

```
        amount_cents = int(amount * 100)
        customer_code = order_id.replace("ORDER_", "CUST_")
```

```
        old_result = self.old_system.make_payment(customer_code, amount_cents)
```

```
        return {
            "success": old_result["status"] == "success",
            "transaction": old_result["transaction_id"],
            "amount": old_result["amount"]
        }
```

### **builder.py**

```
class OrderBuilder:
```

```
    def __init__(self, order_id: str):
```

```
        self.order_id = order_id
        self.customer = ""
        self.items = []
        self.total = 0.0
```

```
    def set_customer(self, customer: str):
```

```
        self.customer = customer
        return self
```

```
    def add_item(self, name: str, price: float, quantity: int = 1):
```

```

self.items.append({"name": name, "price": price, "quantity": quantity})
self.total += price * quantity
return self

```

```

def build(self):
    if not self.customer:
        raise ValueError("Customer is required")
    if not self.items:
        raise ValueError("At least one item is required")
    return {
        "order_id": self.order_id,
        "customer": self.customer,
        "items": self.items,
        "total": self.total
    }

```

### **observer.py**

```

class Order:
    def __init__(self, order_id: str):
        self.order_id = order_id
        self.status = "pending"
        self.observers = []

    def attach(self, observer):
        self.observers.append(observer)

    def set_status(self, status: str):
        self.status = status
        for observer in self.observers:
            observer.update(self.order_id, self.status)

```

```

class EmailNotifier:
    def update(self, order_id: str, status: str):
        print(f"[EMAIL] Order {order_id} is now {status}")

```

### **test\_adapter.py**

```

from adapter import OldPaymentSystem, PaymentAdapter

```

```

def test_adapter():
    old = OldPaymentSystem()
    adapter = PaymentAdapter(old)

```

```

result = adapter.pay("ORDER_100", 150.50)

assert result["success"] is True
assert result["amount"] == 150.50

if __name__ == "__main__":
    print("Running adapter tests...")

    try:
        from adapter import OldPaymentSystem, PaymentAdapter

        old = OldPaymentSystem()
        adapter = PaymentAdapter(old)
        result = adapter.pay("ORDER_100", 150.50)

        assert result["success"] == True
        assert result["amount"] == 150.50
        assert "OLD_" in result["transaction"]
        print("✅ Adapter test - PASSED")
    except Exception as e:
        print(f"❌ Adapter test - FAILED: {e}")

    print("\nAdapter tests completed!")

```

```

===== test session starts =====
collecting ... collected 1 item

test_adapter.py::test_adapter PASSED [100%]

===== 1 passed in 0.18s =====

Process finished with exit code 0

```

### test\_builder.py

```

from builder import OrderBuilder

```

```

def test_builder():
    order = (OrderBuilder("ORDER_001")
            .set_customer("John")
            .add_item("Laptop", 1000)
            .build())

```

```

assert order["order_id"] == "ORDER_001"
assert order["customer"] == "John"
assert order["total"] == 1000

def test_builder_validation():
    builder = OrderBuilder("ORDER_002")

    try:
        builder.build()
        assert False, "Should raise error"
    except ValueError:
        pass

if __name__ == "__main__":
    print("Running builder tests...")

    # Test 1
    try:
        from builder import OrderBuilder

        order =
OrderBuilder("ORDER_001").set_customer("John").add_item("Laptop",
1000).build()
        assert order["order_id"] == "ORDER_001"
        assert order["customer"] == "John"
        assert order["total"] == 1000
        print("✅ Test 1: Basic order - PASSED")
    except Exception as e:
        print(f"❌ Test 1: Basic order - FAILED: {e}")

    # Test 2
    try:
        from builder import OrderBuilder

        builder = OrderBuilder("ORDER_002")
        builder.build()
        print("❌ Test 2: Validation - FAILED (should raise error)")
    except ValueError:
        print("✅ Test 2: Validation - PASSED")

    print("\nBuilder tests completed!")

```

```

if __name__ == "__main__":
    print("Running observer tests...")

    try:
        from observer import Order, EmailNotifier

        order = Order("ORDER_200")
        notifier = EmailNotifier()
        order.attach(notifier)
        order.set_status("paid")
        print("✅ Observer test - PASSED (no errors)")
    except Exception as e:
        print(f"❌ Observer test - FAILED: {e}")

    print("\nObserver tests completed!")

```

```

Running builder tests...
✅ Test 1: Basic order - PASSED
✅ Test 2: Validation - PASSED

Builder tests completed!
Running observer tests...
[EMAIL] Order ORDER_200 is now paid
✅ Observer test - PASSED (no errors)

Observer tests completed!

```

### test\_observer.py

```

from observer import Order, EmailNotifier

```

```

def test_observer():
    order = Order("ORDER_200")
    notifier = EmailNotifier()

    order.attach(notifier)
    order.set_status("paid")
    # Просто проверяем что нет ошибок

```

```
===== test session starts =====
collecting ... collected 1 item

test_observer.py::test_observer PASSED [100%][EMAIL] Order ORDER_200 is now paid

===== 1 passed in 0.11s =====

Process finished with exit code 0
```